TEXORA · LECTURE SHEET

Reference Language: C

The C language as this course uses it: its history and program structure, data types, operators and control statements, and five complete programs — plus a temperature conversion — whose inputs and outputs can be checked line by line.

CSE-2105 · FUNDAMENTALS OF COMPUTING · TOPIC 6 · LEVEL 2 TERM 1

1. A short history of C

C was created by **Dennis Ritchie** at **Bell Labs** in the United States in **1972**. It grew out of an earlier language called B, which itself came from BCPL; the step from B to C added a proper system of data types, which is what made C practical for large programs. C was developed alongside the UNIX operating system, and UNIX was then rewritten in C — a bold proof that a high level language could do work previously reserved for assembly language. Because the same C program could be compiled for many different machines, C spread through universities and industry during the 1980s. In 1989 the American National Standards Institute published a standard version, **ANSI C** (adopted internationally as C90), so that a program written to the standard would behave the same way on every conforming compiler. Later standards (C99, C11, C17 and beyond) added refinements, but the core language in this sheet is the stable ANSI C foundation on which C++, Java, C# and many other languages were later modelled.

C is a good reference language for a first course for three reasons: it is small enough to learn completely at this level, it shows clearly how a program meets the machine (memory, types, addresses), and its control structures — `if`, `for`, `while` — appear in nearly the same form in most languages written after it.

2. Structure of a C program

Every C program in this course follows the same skeleton. The parts, in order, are:

1. **Preprocessor directives** at the top, beginning with #. The commonest is `#include <stdio.h>`, which brings in the standard input and output declarations so that `printf` and `scanf` can be used.
2. **The `main` function.** Execution always begins here. In this course we write it as `int main(void)`: it returns an integer to the operating system and takes no arguments.

3. **The body**, enclosed in braces `{ }`. Inside are declarations (naming the variables and their types) and statements (the actions), each statement ending with a semicolon.
4. `return 0;` as the last statement, reporting to the operating system that the program finished successfully.

Two habits protect beginners. First, the compiler ignores extra spaces and blank lines, so use indentation to show structure — indent everything inside a pair of braces by the same amount. Second, comments — text between `/*` and `*/` — are for human readers only; the compiler discards them, so they are the right place to explain *why*, not to repeat *what*.

3. Character set, tokens, keywords and identifiers

The **character set** of C is the collection of symbols from which programs are built: the letters A–Z and a–z, the digits 0–9, and special characters such as `+ - * / % = < > ( ) { } ; , . " ' # _` and white space. C is **case sensitive**: `Sum`, `sum` and `SUM` are three different names.

The smallest meaningful units of a program are called **tokens**. C has six kinds: keywords, identifiers, constants, string literals, operators and separators (punctuation such as `;`, `,` and braces). A **keyword** is a word reserved by the language with a fixed meaning — `int`, `float`, `if`, `else`, `for`, `while`, `return`, `switch`, `case`, `break`, `void`, `char`, `double`, `const` and others. Keywords cannot be used as variable names. An **identifier** is a name the programmer chooses, for a variable, a function or other program element. The rules for identifiers are strict:

- An identifier may contain letters, digits and the underscore character _ only.
- It must begin with a letter or an underscore, never with a digit: `mark2` is valid, `2mark` is not.
- It must not be a keyword, and it must not contain spaces or other special characters: `total mark` and `total-mark` are both invalid.
- Choose names that say what the thing is: `total_marks` will still make sense in six months; `x7` will not.

4. Data types

A variable's **data type** fixes three things: how much memory it occupies, what values it can hold, and which format specifier is used to print or read it. The four basic types needed here are:

TYPE	SIZE	TYPICAL RANGE OR PRECISION	SPECIFIER	USED FOR
`char`	1 byte	A single character; stored as a code from −128 to 127 (signed)	`%c`	Letters, digits as symbols, grades

`int`	4 bytes	Whole numbers from −2,147,483,648 to 2,147,483,647	`%d`	Counts, marks, sums of whole numbers
`float`	4 bytes	Real numbers with about 6 to 7 significant decimal digits	`%f`	Measurements, averages, prices
`double`	8 bytes	Real numbers with about 15 significant decimal digits	`%lf`	Calculations needing more precision

Sizes are the typical values on a modern 32-bit or 64-bit compiler; the standard guarantees only minimum ranges, so a careful answer says “typically”. The range of a 4-byte `int` follows from its bits: 32 bits with one sign bit give 2^{31} = 2,147,483,648 values on the negative side, and one less on the positive side because zero takes one place. Overflow is silent in C: adding 1 to the largest `int` does not produce an error message, it produces a wrong (wrapped) value — a logical error of exactly the kind described in Topic 5.

5. Variables and constants

A **variable** is a named area of memory whose value can change during the program. It must be **declared** before use, which reserves the memory and fixes the type: `int count;` declares an integer variable named `count`. A declaration may also give a starting value, called initialisation: `int count = 0;`. A variable that is never given a value holds whatever was left in that memory — using it is a classic source of unpredictable results, so initialise as a habit. **Assignment** uses the equals sign as an instruction, not as a mathematical claim: after `count = count + 1;` the variable holds one more than before.

A **constant** cannot change while the program runs. Constants appear as literal values written in the code (`100`, `3.1416`, `'A'`, `"text"`) and as named constants declared with `const`, for example `const float PI = 3.1416f;`. Naming a constant makes its meaning clear and lets a later change be made in exactly one place.

6. Operators

GROUP	OPERATORS	NOTES
Arithmetic	`+ − * / %`	% is the remainder (modulus) operator and works on integers: 17 % 5 = 2. Integer

		division also discards the remainder: 17 / 5 = 3.
Relational	`> < >= <= == !=`	Compare two values and give a true or false result. Note `==` for comparison, against single `=` for assignment — confusing them is a famous beginner trap.
Logical	`&& || !`	Combine conditions: AND needs both true, OR needs at least one true, NOT reverses a condition.
Increment and decrement	`++ --`	Add or subtract 1: `i++` means `i = i + 1`. In a standalone statement the prefix and postfix forms behave the same.
Assignment	`= += -= *= /=`	`sum += mark` is the short form of `sum = sum + mark`.

When operators are mixed, C applies a fixed order of precedence — multiplication and division before addition and subtraction, for example — and brackets override it. In this course, when there is any doubt, add brackets: they cost nothing and remove all argument about what the expression means.

7. Input and output: printf and scanf

The function `printf` writes formatted output. Its first argument is a string that may contain **format specifiers** — placeholders such as `%d` — and the matching values follow as further arguments: `printf("Sum = %d", sum);` prints the words, the value of `sum` in place of `%d`, and nothing else. The characters `\n` inside a string mean “new line”. The function `scanf` reads typed input using the same specifiers, and each variable must be passed with an ampersand, `&`, which gives `scanf` the variable’s address — the place to store what is typed: `scanf("%d", &age);`. Forgetting the `&` is one of the most common errors in first programs. To print a `float` with two decimal places, write `%.2f`; the `.2` fixes the digits after the decimal point.

8. Control statements

C provides the same three control structures as Topic 5. **Selection** uses `if ... else`, whose condition goes in brackets and whose blocks go in braces; for choosing among many fixed alternatives, `switch` compares one integer or character value against a list of `case` labels, with `break` ending each case and `default` catching anything unmatched. **Iteration** offers three loops. The `for` loop

gathers initialisation, condition and update in one line — ideal when the count is known in advance. The `while` loop tests its condition first and may run zero times. The `do ... while` loop tests at the end and therefore always runs at least once, which suits menus and repeated input. Choosing between them is a matter of which reads most clearly for the job; all three can express the same repetition.

9. Arrays and functions, briefly

An **array** stores many values of one type under one name, reached by an index that starts at 0: `int marks[5];` declares space for five integers, `marks[0]` to `marks[4]`. Arrays pair naturally with `for` loops, which visit the elements in order. A **function** is a named block of code that does one job and can be called from many places; it may receive values as arguments (parameters) and may send one value back with `return`. `main` itself is a function, and library functions such as `printf` are functions written by others. Breaking a program into small functions, each testable on its own, is the practical step from classroom programs to real software.

10. Five complete programs

Each program below is complete and standard: it can be typed exactly as shown, compiled, and run. Read each one in three passes — the explanation, the code, then the output — and check the output by hand.

PROGRAM 1 — HELLO, WORLD!

The traditional first program: one output statement inside `main`. `\n` ends the line after printing.

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

Output:

```
Hello, World!
```

PROGRAM 2 — ADD TWO INTEGERS

Two integers are read with one `scanf` call, added into a third variable, and printed. When the user types `12 8` the variables receive 12 and 8 in order.

```
#include <stdio.h>

int main(void)
{
    int a, b, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Sample run (user input shown after the prompt):

```
Enter two integers: 12 8
Sum = 20
```

Check: 12 + 8 = 20 ✓.

PROGRAM 3 — EVEN OR ODD

A number is even exactly when its remainder on division by 2 is zero, which the modulus operator `%` tests directly.

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is even.\n", n);
    else
        printf("%d is odd.\n", n);
    return 0;
}
```

Sample run:

```
Enter an integer: 7
7 is odd.
```

Check: 7 % 2 = 1, not 0, so the `else` branch runs ✓.

PROGRAM 4 — FACTORIAL WITH A FOR LOOP

The accumulator `fact` starts at 1 and is multiplied by every integer from 1 up to `n` — the loop form of the algorithm traced in Topic 5. Here `n` is fixed at 5 in the code, so no input is needed.

```
#include <stdio.h>

int main(void)
{
    int n = 5;
    int i;
    long fact = 1;
    for (i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    printf("Factorial of %d = %ld\n", n, fact);
    return 0;
}
```

Output:

```
Factorial of 5 = 120
```

Check: 1 × 2 × 3 × 4 × 5 = 120 ✓. The type `long` with specifier `%ld` gives the result more room than a plain `int`, because factorials grow very fast.

PROGRAM 5 — LARGEST OF THREE NUMBERS

The first number is taken as the largest so far; each other number replaces it only if it is bigger. This “running champion” pattern solves every largest value problem, for three numbers or for a thousand.

```
#include <stdio.h>

int main(void)
{
    int a = 12, b = 25, c = 9;
```

```
    int largest = a;
    if (b > largest)
        largest = b;
    if (c > largest)
        largest = c;
    printf("Largest = %d\n", largest);
    return 0;
}
```

Output:

```
Largest = 25
```

Check by hand: largest starts at 12; 25 > 12 so largest becomes 25; 9 is not greater than 25, so 25 stands ✓.

WORKED SNIPPET — CELSIUS TO FAHRENHEIT

The conversion formula is:

$$F = C \times 9/5 + 32$$

```
#include <stdio.h>

int main(void)
{
    float celsius = 37.0f;
    float fahrenheit;
    fahrenheit = celsius * 9.0f / 5.0f + 32.0f;
    printf("%.1f C = %.1f F\n", celsius, fahrenheit);
    return 0;
}
```

Output:

```
37.0 C = 98.6 F
```

Check: 37 × 9 = 333; 333 / 5 = 66.6; 66.6 + 32 = 98.6 ✓. The decimal points in `9.0f` and `5.0f` matter: in C, `9 / 5` with integers would give 1, not 1.8, because integer division discards the fraction — the formula must be written with real number constants when the variables are real.

Glossary

TERM	MEANING
Keyword	A word reserved by the C language with a fixed meaning, such as int, if or return; it cannot be used as a name.
Identifier	A programmer chosen name for a variable, function or other element, following the naming rules of Section 3.
Token	The smallest meaningful unit of a program: a keyword, identifier, constant, string, operator or separator.
Declaration	The statement that names a variable and fixes its type before use, such as int count;
Format specifier	A placeholder in a printf or scanf string, such as %d or %f, that states the type of the value handled.
Modulus	The remainder operator %: a % b gives what is left after dividing a by b as integers.
Array	A set of values of one type stored under one name, reached by an index starting at 0.
Function	A named block of code that does one job, can receive arguments and can return a value.

Key points & summary

- C was written by Dennis Ritchie at Bell Labs in 1972, developed from B, and standardised as ANSI C in 1989.
- Every program starts in int main(void), ends statements with semicolons, groups code in braces, and finishes with return 0;
- Identifiers start with a letter or underscore, contain only letters, digits and underscores, and are never keywords; C is case sensitive.
- Typical sizes: char 1 byte (%c), int 4 bytes (%d), float 4 bytes (%f), double 8 bytes (%lf).
- scanf needs the address of each variable — the & — and integer division discards remainders, so formulas with fractions need real constants.
- The five practice programs each verify by hand: 12 + 8 = 20, 7 is odd, 5! = 120, the largest of 12, 25, 9 is 25, and 37 degrees Celsius is 98.6 degrees Fahrenheit.

Practice

Answer in your own words; brief model answers follow.

1. State who created C, where, in which year, and from which earlier language it directly developed.
2. Which of these are valid C identifiers, and why? (a) total_marks (b) 1st_mark (c) total marks (d) float (e) Mark2
3. A variable must hold a temperature to one decimal place, and another must hold the number of students in a class. Give a suitable type and format specifier for each.
4. What is printed by printf("%d", 17 % 5); and by printf("%d", 17 / 5); ? Explain the difference in one sentence each.
5. In Program 3, what output appears if the user enters 24? Show the remainder calculation that decides the branch.
6. A float variable named fahr holds a Fahrenheit value. Write one statement that declares a float named celsius and stores the converted value, using the reverse of the formula in this sheet. Then calculate the result for fahr = 98.6.

Answer key

1. Dennis Ritchie, at Bell Labs, in 1972; C developed directly from the language B.
2. (a) Valid — letters and an underscore, starting with a letter. (b) Invalid — begins with a digit. (c) Invalid — contains a space. (d) Invalid — float is a keyword. (e) Valid — letters and a digit, starting with a letter; note it is different from mark2 because C is case sensitive.
3. Temperature: float (or double) with %f. Number of students: int with %d, because a count is a whole number.
4. 17 % 5 prints 2 — the remainder after dividing 17 by 5. 17 / 5 prints 3 — integer division keeps only the whole part of the quotient and discards the remainder.
5. “24 is even.” — because 24 % 2 = 0, so the condition n % 2 == 0 is true and the first branch runs. (Note the trap for your own code: = assigns a value, while == compares two values.)
6. float celsius = (fahr - 32.0f) * 5.0f / 9.0f; For fahr = 98.6: 98.6 − 32 = 66.6; 66.6 × 5 = 333.0; 333.0 / 9 = 37.0, so celsius = 37.0 — the reverse of the worked snippet, as expected.
